

QA TAKE-HOME · SOLUTION WALKTHROUGH

Testing SauceDemo, end to end.

A Playwright automation suite for the checkout & inventory flows, plus a test plan for a hypothetical affiliate-links feature.

Repo tulip-hw **Stack** Playwright · TypeScript **Date** 2026-09-22

Why Playwright + TypeScript

One tool for the whole matrix: three browser engines, built-in failure artifacts, and a type system that catches page-object drift before CI does.

● Pros

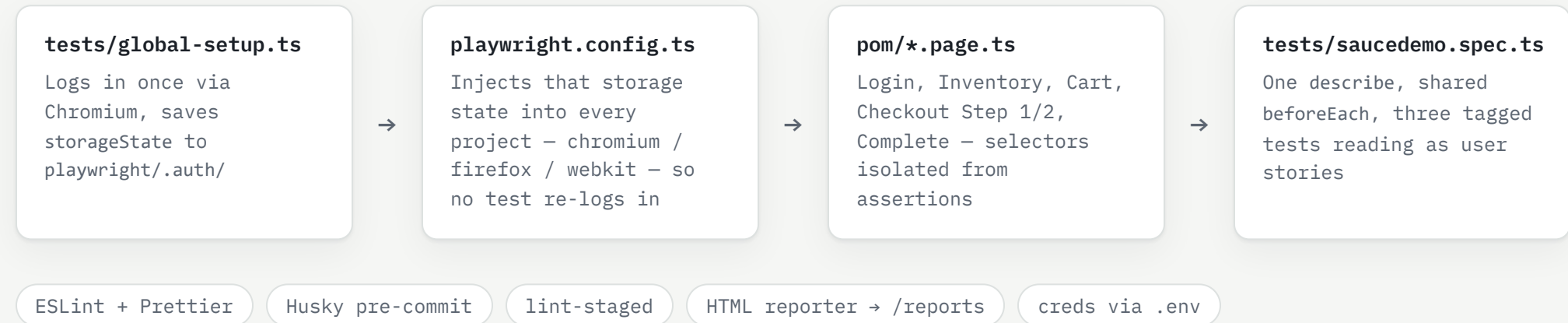
- Auto-waiting locators — no manual `sleep`/polling for elements
- One config, three engines: Chromium, Firefox, WebKit
- Trace, video, and screenshot captured automatically on failure
- Fully parallel by default, fast local + CI runs
- TypeScript keeps page objects and specs in sync at compile time

● Trade-offs

- Younger ecosystem than Selenium — fewer legacy integrations
 - Browser binaries add install/CI image weight
 - JS/TS-first — a cost if the team's stack is elsewhere
 - Codegen output still needs POM discipline applied by hand
-

Project architecture

Auth happens once, selectors live in one place, tests read like the user journey they verify.



Sort logic – @test-3

Each of the four dropdown modes is checked against an order computed fresh from what's actually on the page — not a hardcoded list, so the assertion survives inventory changes and still catches real sort bugs.

tests/saucedemo.spec.ts – @test-3

```
await inventoryPage.selectSortOption('az');
const namesAsc = await inventoryPage.itemNames.allTextContents();
expect(namesAsc).toStrictEqual([...namesAsc].sort((a, b) => a.localeCompare(b)));

const parsePrice = (text: string) => parseFloat(text.replace('$', ''));

await inventoryPage.selectSortOption('lohi');
const pricesAsc = (await inventoryPage.itemPrices.allTextContents()).map(parsePrice);
expect(pricesAsc).toStrictEqual([...pricesAsc].sort((a, b) => a - b));
```

Names – az / za

Read the live text, compare it to that same array run through `localeCompare`. The expectation is derived, not typed in.

Prices – lohi / hilo

Strip the \$, parse to float, compare against a numeric sort — string-sorting "\$9.99" vs "\$10.99" would sort wrong.

Demonstration

Live run at presentation time, or a recorded pass if we're remote.

npm test

Runs all three tagged tests across chromium, firefox, webkit.

npm run test:ui

Playwright's UI mode — step through each test live, watch selectors resolve.

npm run show-report

Opens the HTML report — trace, screenshot, and video attached on any failure.

Test plan – affiliate links

?ref={id} on landing sets an affiliate_tracking cookie, driving attribution, a checkout banner, and backend metrics. Five acceptance criteria, scenarios written positive / negative / edge per AC, then layered across unit, API, E2E, security, manual, and regression.

Representative edge & negative cases

- Empty or malformed ref (?ref=, ?ref=!!@@) — no crash
- Cookie present but **expired** at purchase — treated as no-cookie
- Tampered/unknown influencer id via dev tools
- Duplicate checkout submission — no duplicate metrics record
- Banner text/absence across all five checkout steps

Layer coverage

- **Unit** — ref sanitization, expiry calc, name lookup
- **API** — order payload carries id, metrics write is idempotent
- **E2E** — landing → checkout → banner → completion
- **Security** — cookie tampering, referral fraud
- **Regression** — existing cart/checkout/sort untouched

Questions for product

Every open question below blocks a specific test from having a defined expected result.

OPEN POINT	WHY IT MATTERS FOR TESTING
Does the 30-day expiry reset on every visit, or is it fixed from first set?	Changes the expected cookie value on a second visit test
Multiple ref params on one URL — first wins or last wins?	Defines the expected attribution in a malformed-URL case
Unknown/tampered influencer id server-side — reject or store as-is?	Decides whether that's a negative test or a data-integrity test
Does the banner show fallback text or hide entirely for an unresolvable id?	Defines the expected UI state, not just "don't crash"
Does recorded purchase amount include tax and shipping?	Sets the exact expected value in the metrics assertion
Does setting the cookie require prior consent (GDPR/CCPA)?	Determines whether a consent-gate test is in scope at all

Beyond functional testing

Priority read from impact: metrics accuracy and attribution are P0 because they drive real influencer payouts.

PRI	AREA	REASONING
P0	Backend metrics accuracy	Drives payouts — errors have financial and legal impact
P0	Attribution correctness	Wrong/missing affiliate id misattributes revenue
P1	Banner display	Wrong name or missing banner hurts credibility, not revenue
P1	Cookie duration edges	Silent under/over-attribution if the 30-day logic is off
P2	Security & fraud	Cookie tampering, self-referral, bot traffic — flag as known risk
P2	Privacy / consent	Real compliance risk, likely governed by existing site-wide policy

Accessibility — screen reader, contrast

Responsive — mobile viewport, long names

Cross-browser persistence

Concurrency — race on rapid checkout

Regression on existing checkout